1.Code :

```
from collections import defaultdict, deque

# Graph using adjacency list
graph = defaultdict(list)

def add_edge(u, v):
    graph[u].append(v)
    graph[v].append(u)  # undirected graph

# DFS (Recursive)
def dfs(node, visited):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbor in graph[node]:
            dfs(neighbor, visited)

# BFS (Iterative)
def bfs(start):
    visited = set()
    queue = deque([start])

    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node, end=" ")
            visited.add(node)
            for neighbor in graph[node]:
```

```
            queue.append(neighbor)

# Example graph
add_edge(0, 1)
add_edge(0, 2)
add_edge(1, 3)
add_edge(1, 4)
add_edge(2, 5)

print("DFS Traversal:")
dfs(0, set())

print("\nBFS Traversal:")
bfs(0)
```

Output :

```
 DFS Traversal:
0 1 3 4 2 5
BFS Traversal:
0 1 2 3 4 5
```

2.Code :

```
 import heapq
# Heuristic (Manhattan distance)
def h(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
def astar(grid, start, goal):
    rows, cols = len(grid), len(grid[0])

    open_list = []
    heapq.heappush(open_list, (0, start))
     g = {start: 0}
    parent = {start: None}
      while open_list:
        _, current = heapq.heappop(open_list)

        if current == goal:
            # reconstruct path
            path = []
            while current:
                path.append(current)
                current = parent[current]
            return path[::-1]

        x, y = current
        for dx, dy in [(0,1),(1,0),(0,-1),(-1,0)]:
            nx, ny = x + dx, y + dy
             if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 0:
                new_cost = g[current] + 1
```

```
            if (nx, ny) not in g or new_cost < g[(nx, ny)]:
                g[(nx, ny)] = new_cost
                f = new_cost + h((nx, ny), goal)
                heapq.heappush(open_list, (f, (nx, ny)))
                parent[(nx, ny)] = current
    return None

# Example grid (0 = free, 1 = obstacle)
grid = [
    [0, 0, 0, 0],
    [1, 1, 0, 1],
    [0, 0, 0, 0],
    [0, 1, 1, 0]
]

start = (0, 0)
goal = (3, 3)

path = astar(grid, start, goal)
print("Path:", path)
```

Output :

Path: [(0,0), (0,1), (0,2), (1,2), (2,2), (2,3), (3,3)]

3.Code:

Djikstra's Algorithm

```
import heapq


def dijkstra(graph, start):
    dist = {node: float('inf') for node in graph}
    dist[start] = 0
     pq = [(0, start)]  # (distance, node)


    while pq:
        d, node = heapq.heappop(pq)
        for neighbor, weight in graph[node]:
            new_dist = d + weight


            if new_dist < dist[neighbor]:
                dist[neighbor] = new_dist
                heapq.heappush(pq, (new_dist, neighbor))
    return dist
# Example graph (undirected)
graph = {
    0: [(1, 4), (2, 1)],
    1: [(0, 4), (2, 2), (3, 5)],
    2: [(0, 1), (1, 2), (3, 8)],
    3: [(1, 5), (2, 8)]
}
print("Shortest distances from node 0:")
print(dijkstra(graph, 0))
```

Output : Shortest distances from node 0:

{0: 0, 1: 3, 2: 1, 3: 8}

4.Code :

```
def solve_n_queens(n):
    board = [-1] * n  # board[i] = column of queen in row i

    def is_safe(row, col):
        for i in range(row):
            # same column OR diagonal attack
            if board[i] == col or abs(board[i] - col) == abs(i - row):
                return False
        return True

    def backtrack(row):
        if row == n:
            print(board)  # solution found
            return

        for col in range(n):
            # Branch & Bound: prune invalid choices early
            if is_safe(row, col):
                board[row] = col
                backtrack(row + 1)
                board[row] = -1  # backtrack

    backtrack(0)
# Example
solve_n_queens(4)
```

Output :

[1, 3, 0, 2]

[2, 0, 3, 1]

5.Code :

```
def chatbot():
    print(" Welcome to Customer Support Chatbot!")
    print("Type 'exit' to end the chat.\n")

    while True:
        user = input("You: ").lower()

        if user == "exit":
            print("Bot: Thank you! Have a great day ")
            break

        elif "hello" in user or "hi" in user:
            print("Bot: Hello! How can I assist you today?")

        elif "order" in user:
            print("Bot: Please provide your order ID to check status.")

        elif "price" in user or "cost" in user:
            print("Bot: Our product prices range from ₹500 to ₹5000.")

        elif "refund" in user:
            print("Bot: Refunds are processed within 5-7 business days.")

        elif "delivery" in user:
            print("Bot: Delivery usually takes 3-5 days.")

        elif "bye" in user:
            print("Bot: Goodbye! Visit again ")
```

```
        break
else:
        print("Bot: Sorry, I didn't understand. Can you rephrase?")
# Run chatbot
chatbot()
```

Output :

```
Output
🤖 Welcome to Customer Support Chatbot!
Type 'exit' to end the chat.

You: hello
Bot: Hello! How can I assist you today?
You: order
Bot: Please provide your order ID to check status.
You: price
Bot: Our product prices range from ₹500 to ₹5000.
You:
```